

CNN and MLP for Music Classification

Motivation

Our group is aiming to classify audio clips into musical genres based not only on .wav files, but also based on images of audio waves. With the rise of AI mechanisms in streaming platforms like Spotify and Apple Music, optimizing user experiences is easier than ever. Having algorithms that can classify music into different genres and make song recommendations based on what a user's favorite genre might be is a powerful tool for user retention, specifically from a business standpoint. Genre classification is also a very helpful tool for online music libraries, which can use a model like this to organize their musical collections. The problem our group will be solving is streamlining genre classification for audio clips, with an end goal of classifying genres for newly released music (in the form of .mp3 and .m4a files). We predict that our audio model may outperform our image model for our test set because of increased model complexity and the flexibility that numerical features allow for in multi-layer perceptron neural networks.

Dataset

We will be using the [GTZAN – Music Genre Classification dataset](#) for training, which is commonly used for machine learning research in audio analysis. The dataset contains 1,000 audio tracks across 10 genres, including Blues, Classical, Country, Disco, Hip-hop, Jazz, Metal, Pop, Reggae, and Rock. Each track is 30 seconds long, sampled at 22,050 Hz, resulting in an average length of approximately 662,000 audio samples per track with a standard deviation of about 1.78k samples. Several audio features are extracted from each track for analysis. For example, the chroma STFT mean, which represents how prominent different musical notes are in the audio signal, has an average value of 0.38 and a standard deviation of 0.08. Additionally, the variation in musical note patterns is small, with a chroma variance averaging at 0.09. The root mean square (RMS) energy showed moderate differences in volume across tracks, with a mean of 0.13 and standard deviation of 0.07. There are also various frequency-based features: the spectral centroid, which reflects higher-pitched frequencies of sound, averages about 2200 Hz (moderate pitch); the spectral bandwidth, measuring the range of frequencies, averages roughly 2240 Hz, which means the dataset contains both low and high notes. In addition to the audio tracks, the dataset includes 100 Mel-spectrograms per genre in .png format, totaling 1,000 images, that each represent its corresponding audio file.

We created our own test dataset that includes 100, 30-second-long audio tracks (in MP3 and MPEG-4 formats) from the same genres included in the GTZAN dataset, along with 100 researcher-generated mel-spectrograms to correspond with the audio tracks.

Please reference our Github for our codes: <https://github.com/marissa-data/ml-III-project/tree/main>

Related Work

Much work has been done in this area by past scholars, both with this specific dataset and in general. Meenakshi and Vishnupriya (2018) used a deep learning approach to classify music files into genres, and their approach reached an accuracy of 76%. Similarly to us, Meenakshi and Vishnupriya were motivated by the rapid and increased development of music tracks and the need for them to be accurately classified for easier storage and retrieval. Engelkes (2024) has also done work recently for music genre classification. He also had similar motivations, as he was interested in classifying music to meet the needs of streaming platforms as new songs are developed at an increasing rate. He used a Bottom-up Broadcast Neural Network, in an attempt to preserve “lower-level information”, and he also used enhanced RNN architecture, the Independent Recurrent Neural Network, to look at the sequential orders of the data. Engelkes wanted to use different architectures to satisfy the fact that criteria for music classification changes very often. While there is more work in this area, we found that these two works had similar motivations for classification as us.

Technical and Experimental Approaches

We built two separate deep learning models. We first built a CNN inputting mel-spectrograms and outputting the classified genre. We used cross entropy for both the networks to calculate our loss because we are doing categorical classification.

For our first model we trained a CNN that used spectrograms to classify songs to their corresponding genres. When creating the initial model, we trained, validated, and tested our model’s performance on GTZAN’s instrumental data. Through experimentation with the number of layers, adding patience, and increasing training time, we settled on an initial model with 12 layers: 3 convolution, 3 batch normalization, 3 max pooling, a drop out, an global average pooling, and a fully connected layer. We added patience to help prevent over-fitting, and we increased our epochs to 30, the highest we could repeatedly experiment and modify with the amount of time that the model took to run. We used the Adam optimizer in this model. This model had a 63.8% test accuracy on the instrumental test set. Initially, our data transforms only

included converting the images to tensors, resizing the images to be the same size, and normalizing the mean and standard deviation of the images.

After creating our real world test set, we applied our initial model to it. We used the same initial transforms for the data. However, the initial model had only a 10% accuracy on the final test set. This means that the model performed the same as random guessing (since we only had 10 classes). For our next experiment, we added 3 more layers and trained for longer (50 epochs). We added another set of convolution, batch normalization, and max pooling layers. When retraining the model, the accuracy on the instrumental test set improved significantly to 73%; however, performance became worse on the final test set, as accuracy dropped to 6%. The deeper model was most likely overfitting, as while it performed well on instrumental data, it failed to generalize to real songs with lyrical components. We reverted back to our original general model, and instead decided to add a step to the data pre-processing. We converted images in both the instrumental and final test sets to be gray scale with 3 channels. Adding this step kept the instrumental test accuracy at 60%, but it increased the accuracy of the final test set to 16%. As a final test, we only converted the images in the instrumental data to greyscale to see if that would help the model generalize better, and by doing so, we were able to increase our accuracy to 17%. We kept this model as our final model. When looking at how the model classified within each class, we found that accuracy for rock and country was 70%, for jazz it was 30%, and for the remaining, blues, classical, disco, hip hop, metal, pop, and reggae, the accuracy was 0%. This means that though our model could not classify songs well overall, it excelled at classifying rock and country songs. This could be because rhythmically/instrumentally, those genres don't differ much with or without lyrics. This allows our model to generalize really well for those genres.

The second model we created to classify genres based on raw audio files (instead of images of audio waves) is a multi-layer perceptron (MLP) neural network. In order to extract features of the audio from the .wav files, we used the librosa library in Python to obtain Mel-Frequency Cepstral Coefficients (MFCCs) from each 30-second song clip. This method ensures that different qualities such as tone, spectral shape, and sound frequencies are captured in numerical coefficients. For each 30-second clip, we split up the song into many time frames and generated 40 coefficients for each time frame, creating a time series of features across each song. We then compiled MFCCs and genres for each song into a dataframe for training. Since the genres were all represented in the float datatype, we one-hot encoded genre labels for

compatibility with our neural network. Afterwards, we implemented a train-validation split to validate our model on 20% of the original data; standard scaling was also conducted on both training and validation sets to ensure optimal performance.

To create the MLP genre classification network, we started with a sequential model and added four dense layers (an input layer, two hidden layers, and an output layer) with ReLU activations in the first three layers and softmax activation in the output layer to start; all of this was accomplished using keras tools within the tensorflow library in Python. After compiling our model with an Adam optimizer, cross-entropy loss, and an accuracy metric, we fit our model on our training data with 100 epochs and a batch size of 32.

Before tuning our hyperparameters in the neural network, our validation set accuracy was at about 45% (using a random seed to track changes in performance). The first fix we made to improve accuracy was to implement early stopping with a patience of 10, which improved genre classification by about 5% (about 50% accuracy). Looking into how we could improve each of the MLP network layers, we decided to add L2 regularization on the second hidden layer, which improved model performance by about 2% (about 52% accuracy). After doing more research on MFCCs, we decided to update our original MFCC extraction to summarize features across time frames for each song through mean and standard deviation, creating instead one fixed-length (40 means + 40 standard deviations = 80 coefficients) feature vector per song. Our validation set accuracy then went all the way up to 69% because our model learned better when song features were summarized as opposed to when they were in a time series format.

When it came to running our test dataset through our MLP network, we ran into a few issues. First of all, our test dataset includes raw audio files in .mp3 and .m4a formats, not .wav. But since we are putting extracted MFCC features through our model, the raw audio structures should not impact performance. We were able to alter our pre-processing pipeline to account for differing file formats. In trying out different MFCC extraction methods (to improve both validation and test accuracy), we were able to get the testing accuracy from 25% to 31% when switching from a time series of MFCC features within each song to means and standard deviations of each feature. Additionally, updating our MFCC code to normalize the new audio files improved testing accuracy from 31% to 33%. In order to prevent overfitting on the training dataset, we did not include these features in our final model. Lastly, upon further inspection specifically by genre, we discovered that our model accurately classified classical, hip-hop, and

reggae music (above 60% accuracy) more often than blues and metal (about 40% accuracy), country, rock, and jazz (about 20% accuracy), and disco and pop (less than 10% accuracy) in our test data.

Because of relatively poor overall accuracy on real-world music, we decided to build upon the model further to make it more generalizable. We split the audio files into 10 3-second segments to provide more training data for the model, as well as extracting more MFCCs from each segment. We also added weights to all genre classes and manually coded a higher weight to poorer performing genres, such as pop. We also used less dense layers with more neurons, added an L2 penalty, and reduced the learning rate during training if the model got stuck on a loss value. We found that during training the model, it began to plateau at a validation loss of ~ 1.2 . Even with using different learning rates, we were unable to get the model out of the plateau. This, unfortunately, indicated to us that the model most likely was unable to learn any further. Luckily, the training and validation losses were close in value, indicating that the model was not overfitting to the training set.

The final validation accuracy for this model on the GTZAN data was 68%, and the final test accuracy on real-world songs was 41%. This model performed more accurately on real-world music than the previous, however, the model was extremely biased towards the hip hop class. Accuracy for hip hop was 99%, while the accuracy for pop was 0%. Metal, classical, and reggae were predicted at above 50% accuracy; blues, disco, and jazz were predicted at above 20% accuracy; country and rock were predicted at above 1% accuracy.

Conclusion

Overall, we were able to get a model to perform at a 41% accuracy on modern music, with higher accuracy per each genre in all three models created during the project timeline. We learned through this project that our models could not generalize enough to accurately predict real-world music genres. This makes it clear that the GTZAN data does not have accurate genre applicability to modern music. Intuitively, this is consistent with the logic that train and test data should be similar enough for a model to perform accurately with test data. A great future step for this model would be to use real-world music as both the train and tests to accurately predict modern genres today. We also learned that music has evolved greatly over the past 30 years that a model would be unable to accurately predict genres (without professional subject matter expertise).

References

- Engelkes, J. (2024). Music Genre Classification Capabilities of Enhanced Neural Network Architectures. *Scholarly Horizons: University of Minnesota, Morris Undergraduate Journal*, 11(1). <https://doi.org/10.61366/2576-2176.1135>
- Vishnupriya, S., & Meenakshi, K. (2018). Automatic Music Genre Classification using Convolution Neural Network. *2018 International Conference on Computer Communication and Informatics (ICCCI)*. <https://doi.org/10.1109/iccci.2018.8441340>